TRƯỜNG/VIỆN: **Trường Công nghệ Thông tin và Truyền thông - ĐHBKHN**

KHOA/BỘ MÔN: **Khoa học máy tính**

HỌC PHẦN: **Khai phá Web**

ĐỀ XUẤT DỰ ÁN (PROJECT PROPOSAL)

Adaptive Retrieve–Fuse–Re-rank Search Engine Reinforcement Learning Query Reformulation

Giải quyết Bounded Recall trong kiến trúc tìm kiếm đa tầng

Mục	**Thông tin**
Đề tài	Adaptive Retrieve–Fuse–Re-rank Search Engine RL Query Reformulation
Lĩnh vực	Information Retrieval, Reinforcement Learning, NLP
Nhóm thực hiện	Đỗ Thành Đức – 20242095M
	Trịnh Quang Quân – 20242556M
	Nguyễn Nhật Minh – 20252184M
	Vũ Việt Anh – 20251047M
Giảng viên hướng dẫn	Dương Quang Huy
Ngày nộp	Ngày 20 tháng 12 năm 2025

Tóm tắt nội dung

Các hệ thống tìm kiếm hiện đại thường dùng kiến trúc đa tầng (retrieval → re-ranking), trong đó tầng retrieval chỉ trả về top-k ứng viên (ví dụ top-100 hoặc top-100). Nếu tài liệu liên quan không nằm trong tập ứng viên này, tầng re-ranker (cross-encoder) sẽ *không bao giờ* nhìn thấy, dẫn đến hiện tượng **bounded recall**. Đề tài này đề xuất Adaptive Retrieve–Fuse–Re-rank Search Engine: một pipeline *Retrieve–Fuse–Re-rank* có cơ chế **Reinforcement Learning (RL) Query Reformulation** để tạo ra nhiều biến thể truy vấn, chạy retrieval song song, hợp nhất kết quả bằng **Reciprocal Rank Fusion (RRF)** và cuối cùng re-rank bằng **BERT Cross-Encoder**. Hệ thống được đánh giá trên MS MARCO Passage Ranking và kiểm tra tính tổng quát ngoài miền (OOD) trên các tập con của BEIR, với mục tiêu tăng **Recall@100** và cải thiện **MRR@10/nDCG@10** trong khi kiểm soát **độ trễ**.

Mục lục

1 Bối cảnh và động lực (Motivation) 1
1.1 Kiến trúc Cascade Ranking và nút thắt Bounded Recall . . . 1
1.2 Tại sao Query Expansion truyền thống chưa đủ? . . . 1

2 Mục tiêu và câu hỏi nghiên cứu 2
2.1 Mục tiêu tổng quát . . . 2
2.2 Câu hỏi nghiên cứu (Research Questions) . . . 2

3 Liên quan công trình (Related Work) 3
3.1 RL query reformulation . . . 3
3.2 Pseudo-Relevance Feedback và RM3 . . . 3
3.3 RRF fusion . . . 3
3.4 BM25 và tầng retrieval . . . 3
3.5 MS MARCO và đánh giá MRR@10 . . . 3
3.6 BERT cross-encoder reranking . . . 3
3.7 BEIR và OOD generalization . . . 4
3.8 Công cụ thực nghiệm . . . 4

4 Đề xuất phương pháp (Methodology) 5
4.1 Tổng quan pipeline . . . 5
4.2 Định nghĩa MDP cho RL Reformulation . . . 5
4.3 Chính sách và thuật toán học . . . 6
4.4 Retrieve & Fuse với RRF . . . 6
4.5 Re-ranking với BERT Cross-Encoder . . . 6

5 Thiết kế thí nghiệm (Evaluation Plan) 7
5.1 Dữ liệu . . . 7
5.2 Metrics . . . 7
5.3 Baselines . . . 7
5.4 Ablation đề xuất (để báo cáo trông như paper IR) . . . 8

6 Kế hoạch triển khai từng bước (Implementation Roadmap) 9
6.1 Bước 1: Thiết lập retrieval baseline (BM25) và đánh giá . . . 9
6.2 Bước 2: Candidate-term mining . . . 9
6.3 Bước 3: Xây agent RL cho reformulation . . . 9
6.4 Bước 4: Multi-query retrieval và RRF . . . 9
6.5 Bước 5: BERT re-ranking . . . 9

7 Rủi ro và phương án giảm thiểu (Risks & Mitigation) 10

8 Timeline và phân công 11

9 Đóng góp dự kiến (Expected Contributions) 12

1 Bối cảnh và động lực (Motivation)

1.1 Kiến trúc Cascade Ranking và nút thắt Bounded Recall

Trong thực tế và trong nhiều benchmark hiện đại, kiến trúc tìm kiếm đa tầng (multi-stage ranking) là tiêu chuẩn: tầng 1 dùng phương pháp rẻ (BM25/dense retriever) để lọc nhanh một danh sách ứng viên; tầng 2 dùng mô hình đắt (cross-encoder) để xếp hạng lại chính xác hơn. **Vấn đề**: nếu tài liệu liên quan không xuất hiện trong top-k ứng viên của tầng 1 thì tầng 2 không thể cứu vãn — đây là *bounded recall*. Điều này đặc biệt nghiêm trọng khi truy vấn ngắn/khó, từ vựng không khớp (vocabulary mismatch), hoặc miền dữ liệu thay đổi.

1.2 Tại sao Query Expansion truyền thống chưa đủ?

Pseudo-relevance feedback như RM3 có thể cải thiện recall nhưng thường mang tính heuristic, nhạy tham số, dễ gây *query drift* (lệch ý định truy vấn) khi top tài liệu ban đầu bị nhiễu. Đề tài này hướng tới **một cơ chế học chính sách** để chọn term mở rộng có điều kiện theo ngữ cảnh, tối ưu trực tiếp theo metric IR thay vì dựa vào quy tắc thủ công.

2 Mục tiêu và câu hỏi nghiên cứu

2.1 Mục tiêu tổng quát

Xây dựng một hệ thống tìm kiếm Adaptive Retrieve–Fuse–Re-rank Search Engine có khả năng **tăng recall của tầng retrieval** bằng **RL query reformulation**, sau đó **hợp nhất** và **re-rank** để tối ưu độ chính xác cuối.

2.2 Câu hỏi nghiên cứu (Research Questions)

1. RL reformulation có giúp tăng **Recall@100** so với RM3 và các baseline cascade chuẩn không?
2. Việc chạy *multi-query retrieval + RRF* có cải thiện ổn định (robustness) trên OOD (BEIR) không?
3. Trade-off giữa **Recall** và **Latency** thay đổi thế nào theo số lượng query reformulations m và ngân sách re-ranking K?
4. Agent học được những kiểu hành động nào (chọn term từ query vs từ tài liệu ứng viên), và chúng đóng góp ra sao?

3 Liên quan công trình (Related Work)

3.1 RL query reformulation

Nogueira & Cho đề xuất một tác nhân (agent) chọn từ/cụm từ từ tập ứng viên để xây dựng truy vấn mới nhằm *tối đa hoá số tài liệu liên quan được truy hồi* (reward thường dựa trên recall) [1]. Điểm quan trọng là quá trình "viết lại truy vấn" được học như một **chính sách** (policy) có điều kiện theo ngữ cảnh, giúp giảm lệ thuộc vào heuristic và đối phó tốt hơn với *vocabulary mismatch*. Cách tiếp cận này là nền tảng cho module RL Reformulation trong đề tài.

3.2 Pseudo-Relevance Feedback và RM3

RM3 xuất phát từ họ mô hình "relevance model", ước lượng phân phối từ vựng của lớp tài liệu liên quan từ top tài liệu truy hồi ban đầu [3]. Sau đó, truy vấn được mở rộng bằng cách trộn (interpolate) mô hình truy vấn gốc với mô hình liên quan ước lượng, giúp tăng khả năng bao phủ từ đồng nghĩa/biến thể. Tuy nhiên, RM3 dễ bị *query drift* nếu top tài liệu ban đầu nhiễu, nên rất phù hợp làm baseline so sánh với RL.

3.3 RRF fusion

Reciprocal Rank Fusion (RRF) là phương pháp hợp nhất nhiều danh sách xếp hạng bằng cách cộng "nghịch đảo thứ hạng" của mỗi tài liệu trong từng list [2]. Vì chỉ dùng rank (không dùng score tuyệt đối), RRF thường bền vững khi kết hợp nhiều hệ retrieval/biến thể truy vấn có thang điểm khác nhau. Trong hệ đề xuất, RRF đóng vai trò tăng recall và giảm dao động giữa các truy hồi multi-query.

3.4 BM25 và tầng retrieval

BM25 là hàm xếp hạng xác suất phổ biến cho lexical retrieval, dựa trên TF-IDF có chuẩn hoá theo độ dài tài liệu [4]. Ưu điểm là nhanh, ổn định, và thường là baseline mạnh ngay cả trong các benchmark đa miền. Trong đề tài, BM25 được dùng làm tầng retrieval chính (và có thể thay thế bằng dense/hybrid để so sánh).

3.5 MS MARCO và đánh giá MRR@10

MS MARCO Passage Ranking là benchmark lớn cho passage ranking/re-ranking với qrels và quy trình đánh giá tiêu chuẩn [5]. Trên leaderboard, metric chính thường là **MRR@10** và định dạng nộp kết quả (TSV qid–pid–rank) kèm script đánh giá chính thức được công bố công khai [6]. Điều này giúp thí nghiệm của đề tài dễ tái lập và so sánh công bằng.

3.6 BERT cross-encoder reranking

Cross-encoder BERT chấm điểm relevance bằng cách đưa *(query, passage)* vào cùng một mô hình và học tương tác token-level, thường cho chất lượng cao hơn bi-encoder nhưng chi phí lớn [7]. Vì vậy, mô hình này phù hợp để re-rank trên top-K ứng viên sau khi fusion, tối ưu chất lượng cuối mà vẫn kiểm soát độ trễ. Đây là tầng "đắt" trong cascade, nên đề tài sẽ báo cáo rõ trade-off chất lượng–latency.

3.7 BEIR và OOD generalization

BEIR là bộ benchmark đa miền (nhiều kiểu truy hồi và domain) được thiết kế để đánh giá khả năng tổng quát (zero-shot/OOD) của các hệ IR [8]. Kết quả trong BEIR thường cho thấy mô hình hoạt động tốt trên in-domain chưa chắc ổn định ngoài miền, đặc biệt khi miền có thuật ngữ chuyên biệt. Vì vậy, việc kiểm tra trên BEIR giúp đề tài đánh giá tính ứng dụng thực tế của RL reformulation và cơ chế fusion.

3.8 Công cụ thực nghiệm

Pyserini/Anserini là toolkit hỗ trợ tái lập thí nghiệm IR với cả sparse và dense retrieval, cung cấp index dựng sẵn, script đánh giá và các pipeline đa tầng [9]. Nhờ đó, nhóm có thể nhanh chóng dựng baseline BM25/RM3, chạy multi-query retrieval và đo latency theo từng stage. Điều này giảm thời gian "engineering" và tập trung vào đóng góp thuật toán.

4 Đề xuất phương pháp (Methodology)

4.1 Tổng quan pipeline

Pipeline đề xuất gồm 4 giai đoạn chính (từ rẻ đến đắt), nhằm tăng recall trước khi re-rank:

1. **Giai đoạn 0 – Candidate-term Mining**: dùng BM25 lấy top-k_0 tài liệu sơ bộ (pseudo-relevant set), sau đó trích xuất term/cụm từ ứng viên C. Thực tế có thể dùng TF-IDF top-n term, thống kê đóng góp BM25, hoặc keyphrase extraction (RAKE/KeyBERT) để có candidates mang tính "ngữ nghĩa" hơn. Tập C được lọc stopwords, loại trùng và giới hạn kích thước (ví dụ $|C| = 50$–200) để RL có không gian hành động vừa đủ.

2. **Giai đoạn 1 – RL Query Reformulation**: agent nhận trạng thái s_t (query gốc q_0, query hiện tại q_t, và candidates C) và chọn hành động a_t (thêm term/phrase hoặc `STOP`). Mục tiêu là học chính sách chọn term tối ưu theo metric IR, thay vì chọn theo tần suất thuần tuý như RM3; đồng thời dùng ràng buộc (phạt độ dài, cấm chọn lặp) để hạn chế query drift. Kết thúc episode, agent sinh ra m biến thể $\{q^{(1)}, \ldots, q^{(m)}\}$ để tăng độ phủ truy hồi.

3. **Giai đoạn 2 – Retrieve & Fuse**: chạy retrieval song song cho từng query $q^{(i)}$ để thu ranked lists L_i. Sau đó hợp nhất bằng RRF để tận dụng tài liệu xuất hiện "tương đối cao" trong bất kỳ list nào, giúp tăng Recall@100/Recall@k. Việc song song hoá và cache top-k_0 ở stage 0 là chìa khoá kiểm soát latency.

4. **Giai đoạn 3 – Neural Re-ranking**: dùng BERT cross-encoder re-rank top-K sau fusion để tối ưu chất lượng ở top head (MRR@10/nDCG@10). Do cross-encoder đắt, đề tài sẽ khảo sát ảnh hưởng của K (ví dụ 50/100/200) và tối ưu suy luận bằng batching và mixed precision.

4.2 Định nghĩa MDP cho RL Reformulation

Ta mô hình hoá quá trình reformulation như một MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

State s_t.

$$s_t = \big(\mathrm{enc}(q_0), \mathrm{enc}(q_t), \mathrm{enc}(C), t\big)$$

Trong đó enc$(\cdot)$ có thể là: (i) bag-of-words + IDF để đơn giản và nhanh; hoặc (ii) embedding từ transformer/bi-encoder để nắm bắt quan hệ ngữ nghĩa. Để agent "biết" term nào hữu ích, state thường bổ sung feature như: độ trùng với query, TF-IDF trong top-k_0, và độ tương đồng embedding giữa term và query.

Action a_t.

$$\mathcal{A} = C \cup \{\texttt{STOP}\}$$

Mỗi hành động chọn 1 term/phrase $c \in C$ để thêm vào q_t (hoặc dừng). Để tránh drift, có thể thêm ràng buộc: không chọn lại term đã dùng, giới hạn độ dài truy vấn, hoặc chỉ cho phép chọn term có cosine similarity vượt ngưỡng với embedding của q_0.

Transition. $q_{t+1} = q_t \oplus a_t$ (nối term vào query), còn C có thể giữ nguyên hoặc cập nhật (ví dụ: loại term đã dùng). Trong các thiết kế thực tế, episode dài T nhỏ (2–5 bước) thường đủ để tăng recall mà không làm query quá dài.

Algorithm 1: Huấn luyện RL Query Reformulation (minh hoạ REINFORCE)

Input: Tập truy vấn huấn luyện $\mathcal{Q}$, retrieval engine $\mathcal{E}$, qrels, số bước tối đa T
for *epoch* $= 1..E$ **do**
  foreach $q_0 \in \mathcal{Q}$ **do**
    Lấy top-k_0 bằng $\mathcal{E}(q_0)$, trích xuất candidates C
    $q \leftarrow q_0$; lưu trajectory $\tau \leftarrow \emptyset$
    for $t = 1..T$ **do**
      Sample $a_t \sim \pi_\theta(\cdot|s_t)$
      if $a_t =$ *STOP* **then**
        break
      $q \leftarrow q \oplus a_t$
      Lưu (s_t, a_t) vào τ
    Chạy retrieval cho q và tính reward $R(\tau)$ theo qrels
    Cập nhật $\theta \leftarrow \theta + \eta \nabla_\theta \log \pi_\theta(\tau) \cdot (R(\tau) - b)$

Reward. Để gắn trực tiếp với mục tiêu IR và tránh reward quá thưa, đề xuất reward theo cải thiện metric:

$$r = \Delta\text{MRR@10} = \text{MRR@10}(q') - \text{MRR@10}(q_0)$$

Ngoài ra dùng **reward shaping** để học ổn định hơn:

$$r = \alpha \Delta\text{Recall@100} + (1-\alpha)\Delta\text{MRR@10} - \lambda \cdot |q'|$$

Trong thực nghiệm, Recall@100 thường "mượt" hơn MRR@10 nên hữu ích để huấn luyện policy ở giai đoạn đầu, sau đó có thể tăng trọng số MRR@10 để tối ưu chất lượng top head.

4.3 Chính sách và thuật toán học

Dùng policy gradient (REINFORCE) hoặc actor-critic; agent output phân phối $\pi_\theta(a_t|s_t)$. Để giảm phương sai gradient, sử dụng baseline b (moving average hoặc value network) và chuẩn hoá reward theo batch.

4.4 Retrieve & Fuse với RRF

Với m ranked lists $L_1, \ldots, L_m$, điểm RRF cho tài liệu d:

$$\text{RRF}(d) = \sum_{i=1}^{m} \frac{1}{k + \text{rank}_i(d)}$$

với k là hằng số làm mượt (thực hành thường dùng $k = 60$). RRF phù hợp vì không cần chuẩn hoá score giữa các retrieval runs và thường hoạt động tốt khi mỗi list cung cấp "một góc nhìn" khác nhau từ các query reformulations.

4.5 Re-ranking với BERT Cross-Encoder

Cross-encoder chấm điểm bằng cách mô hình hoá tương tác sâu giữa query và passage, thường cải thiện đáng kể MRR@10/nDCG@10 so với chỉ retrieval [7]. Do chi phí tính toán cao, hệ thống chỉ re-rank top-K sau fusion và đo latency chi tiết để tìm cấu hình cân bằng. Nếu thời gian cho phép, có thể thử distillation (miniLM) để tăng throughput nhưng vẫn giữ chất lượng.

5 Thiết kế thí nghiệm (Evaluation Plan)

5.1 Dữ liệu

In-domain (MS MARCO). Sử dụng MS MARCO Passage Ranking (train/dev) với qrels chuẩn và quy trình đánh giá phổ biến trong IR hiện đại [5]. Metric chính là **MRR@10**; đồng thời bộ công cụ chính thức cung cấp script đánh giá và định dạng output thống nhất (TSV qid–pid–rank) [6]. Điều này giúp so sánh công bằng giữa baseline (BM25/RM3) và hệ đề xuất.

Out-of-domain (BEIR). Chọn 3 tập con từ BEIR (ví dụ: NFCorpus, SciFact, FiQA) để đo khả năng tổng quát ngoài miền [8]. Các tập này khác biệt về ngôn ngữ chuyên ngành và kiểu truy vấn, giúp kiểm tra độ bền vững của RL reformulation và cơ chế fusion. Kết quả OOD được báo cáo cùng phân tích lỗi (truy vấn drift, term không phù hợp, thiếu từ vựng miền).

5.2 Metrics

- **MRR@10**: đo chất lượng xếp hạng ở top 10; nhạy với việc đưa đúng tài liệu liên quan lên sớm (phù hợp với search). Vì đây là metric chuẩn của MS MARCO Passage Ranking, thí nghiệm sẽ ưu tiên tối ưu MRR@10 và so sánh theo script chính thức.
- **nDCG@10**: phản ánh chất lượng xếp hạng có tính đến mức độ liên quan (graded relevance) nếu có; hữu ích để so sánh khi có nhiều tài liệu liên quan.
- **Recall@100**: đo bounded recall ở tầng retrieval/fusion (tài liệu liên quan có nằm trong top-100 hay không). Đây là chỉ số then chốt để chứng minh module reformulation + fusion có "mở rộng" được candidate set cho re-ranker.
- **Latency (ms/query)**: báo cáo theo từng stage (candidate mining, RL step, multi-query retrieval, fusion, re-ranking). Mục tiêu là chỉ ra trade-off giữa tăng recall và chi phí thời gian khi tăng m hoặc K.

5.3 Baselines

1. **BM25 (first-stage)**: baseline lexical retrieval nhanh và ổn định [4]. Đây là điểm xuất phát để đo bounded recall và làm tầng ứng viên cho re-ranker.
2. **BM25 + RM3**: áp dụng pseudo-relevance feedback để mở rộng truy vấn từ top tài liệu ban đầu [3]. Baseline này thường cải thiện recall nhưng có nguy cơ query drift; vì vậy rất phù hợp để so sánh với RL (học chính sách thay vì heuristic).
3. **BM25 → BERT re-rank**: cascade chuẩn dùng cross-encoder chỉ re-rank top-K từ BM25 [7]. Baseline này cho chất lượng cao khi bounded recall đủ tốt; nếu bounded recall kém thì hiệu quả bị giới hạn.
4. **(Tuỳ chọn) Dense retrieval**: bi-encoder/dense retriever để so sánh bounded recall giữa lexical và dense; sau đó vẫn dùng cùng re-ranker để công bằng. Mục tiêu là xem RL reformulation mang lại lợi ích bổ sung khi tầng 1 đã chuyển sang dense/hybrid.

5.4 Ablation đề xuất (để báo cáo trông như paper IR)

- **Không RL**: thay agent bằng heuristic chọn term (TF-IDF top terms) từ C. Ablation này giúp tách bạch lợi ích đến từ *multi-query* so với lợi ích đến từ *học chính sách chọn term*.
- **Không RRF**: thay fusion bằng union+BM25-score hoặc CombSUM (cần chuẩn hoá). Mục tiêu là kiểm tra RRF có giúp ổn định khi kết hợp nhiều list (khác thang điểm/khác độ dài) hay không.
- **Số biến thể query** m: thử $m \in \{1, 2, 4, 8\}$ để quan sát đường cong Recall–Latency. Thực tế thường có điểm bão hoà: tăng m thêm chỉ tăng recall rất ít nhưng latency tăng tuyến tính.
- **Nguồn candidate terms**: (i) chỉ từ q_0; (ii) chỉ từ top-k_0 docs; (iii) kết hợp. Ablation này kiểm tra liệu candidate mining từ pseudo-relevant docs có giúp vượt vocabulary mismatch tốt hơn hay lại làm drift.
- **Reward**: so sánh reward chỉ Recall@100, chỉ MRR@10, và reward shaping hỗn hợp. Kết quả giúp hiểu metric nào "dạy" agent ổn định nhất và phù hợp mục tiêu bounded recall.

6 Kế hoạch triển khai từng bước (Implementation Roadmap)

Phần này được viết để bạn có thể triển khai *có thể chạy được* theo từng mốc.

6.1 Bước 1: Thiết lập retrieval baseline (BM25) và đánh giá

- Dùng Pyserini/Anserini để index và chạy BM25 trên MS MARCO Passage Ranking [9].
- Dùng script đánh giá chính thức MRR@10 theo hướng dẫn MS MARCO [6].
- Kết quả mục tiêu: tái lập được baseline MRR@10 hợp lý trên dev, kèm báo cáo Recall@100 để nhìn bounded recall.

6.2 Bước 2: Candidate-term mining

- Lấy top-k_0 docs với BM25 (ví dụ $k_0 = 50$ hoặc 100) và gom nội dung để thống kê term/phrase.
- Trích xuất candidates: (i) TF-IDF top-n; (ii) term contribution theo BM25; (iii) RAKE/KeyBERT (tuỳ chọn) để lấy cụm từ có tính ngữ nghĩa.
- Lọc: stopwords, stemming/lemmatization, loại trùng, giới hạn $|C|$ (ví dụ 50–200) để RL có không gian hành động kiểm soát được.

6.3 Bước 3: Xây agent RL cho reformulation

- Khởi tạo policy network: MLP trên features (IDF, term freq, query-term overlap, cosine embedding) hoặc transformer nhẹ nếu đủ tài nguyên.
- Môi trường (environment): "step" = thêm term; "episode" = tạo query mới và chạy retrieval để tính reward theo qrels.
- Dùng REINFORCE + baseline (moving average) hoặc actor-critic để ổn định; log lại hành động để phân tích loại term agent thường chọn.

6.4 Bước 4: Multi-query retrieval và RRF

- Tạo m queries bằng sampling nhiều episode hoặc beam-search trên policy (để giảm trùng lặp giữa các biến thể).
- Chạy retrieval cho từng query (song song) và thu các ranked list; cache để tiết kiệm thời gian khi ablation.
- Fuse bằng RRF; kiểm tra tăng Recall@100 và phân tích tài liệu mới được "kéo" vào candidate set.

6.5 Bước 5: BERT re-ranking

- Re-rank top-K sau RRF bằng cross-encoder BERT; báo cáo MRR@10/nDCG@10 và độ trễ suy luận.
- Tối ưu throughput: batch inference, mixed precision; nếu cần có thể thử model nhẹ hơn (distillation) để giảm latency.

7 Rủi ro và phương án giảm thiểu (Risks & Mitigation)

- **Reward quá thưa/dao động**: dùng shaping (Recall@100 + MRR@10), baseline trong policy gradient, hoặc curriculum (huấn luyện trên query dễ trước).
- **Chi phí retrieval multi-query cao**: giới hạn m, cache kết quả stage 0, song song hoá retrieval, hoặc dùng approximate retrieval.
- **Query drift**: phạt độ dài query, ràng buộc chọn term gần với query intent (semantic similarity).
- **OOD giảm mạnh**: thêm regularization, hoặc dùng candidates dựa trên dense semantic terms.

8 Timeline và phân công

Tuần	Công việc chính
1–2	Dựng BM25 baseline, chạy MRR@10 theo script chính thức; viết báo cáo baseline.
3–4	Candidate mining + RM3 baseline; kiểm tra Recall@100 và query drift.
5–6	Cài RL environment + policy baseline (REINFORCE); chạy thử trên subset.
7–8	Multi-query retrieval + RRF; ablation theo m, k_0, K.
9–10	BERT cross-encoder re-ranking; tối ưu latency; tổng hợp kết quả.
11–12	OOD evaluation trên BEIR; viết báo cáo/paper-style, làm hình và bảng.

9 Đóng góp dự kiến (Expected Contributions)

1. **Giải pháp RL thực dụng** để cải thiện bounded recall trong cascade ranking.
2. **Phân tích trade-off** Recall vs Latency theo ngân sách multi-query và re-ranking.
3. **Đánh giá robustness** trên BEIR (OOD) và phân tích lỗi.
4. **Bộ ablation đầy đủ** và khuyến nghị cấu hình cho hệ multi-stage IR nhỏ gọn.

Tài liệu tham khảo

[1] Rodrigo Nogueira and Kyunghyun Cho. Task-Oriented Query Reformulation with Reinforcement Learning. In *EMNLP*, 2017. https://aclanthology.org/D17-1061/.

[2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. In *SIGIR*, 2009. https://plg.uwaterloo.ca/~gvcormac/cormacksigir09-rrf.pdf.

[3] Victor Lavrenko and W. Bruce Croft. Relevance-Based Language Models. In *SIGIR*, 2001. https://ir.webis.de/anthology/2001.sigirconf_conference-2001.16/.

[4] Stephen Robertson and Hugo Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 2009. https://doi.org/10.1561/1500000019.

[5] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. MS MARCO: A Human Generated Machine Reading Comprehension Dataset. arXiv:1611.09268, 2016. https://www.microsoft.com/en-us/research/publication/ms-marco-human-generated-machine-reading-comprehension-dataset/.

[6] MS MARCO. Submit to Document and Passage Ranking Leaderboards: File format and official evaluation script (MRR@10). 2025. https://microsoft.github.io/msmarco/Submission.html.

[7] Rodrigo Nogueira and Kyunghyun Cho. Passage Re-ranking with BERT. arXiv:1901.04085, 2019. https://arxiv.org/abs/1901.04085.

[8] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. NeurIPS Datasets and Benchmarks, 2021. https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/file/65b9eea6e1cc6bb9f0cd2a47751a186f-Paper-round2.pdf.

[9] Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations. In *SIGIR*, 2021. https://pypi.org/project/pyserini/.